

Học Sâu (Deep Learning)

Chương 7: Lập trình Chuyên sâu với Keras (A Deep Dive on Keras)

Đại học Đông Á

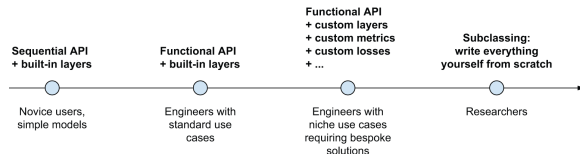
August 14, 2026

Nội dung chương

1. Triết lý Thiết kế của Keras
2. Ba phương pháp Xây dựng Mô hình
3. Tùy biến Vòng lặp Huấn luyện (Customizing)
4. Callbacks: Tự động hóa quá trình Huấn luyện
5. Giám sát với TensorBoard
6. Tổng kết

Từng bước phơi bày sự phức tạp (Progressive Disclosure)

- **Nguyên tắc thiết kế Keras:** Làm cho mọi thứ trở nên dễ bắt đầu, nhưng vẫn hoàn toàn khả thi để xử lý các bài toán siêu phức tạp.
- Không có một "con đường duy nhất". Keras cung cấp một **Phổ quy trình (Spectrum of workflows)**.
- Các trường hợp đơn giản có thể được giải quyết bằng vài dòng mã, trong khi các công việc phức tạp có thể yêu cầu viết mã chi tiết.



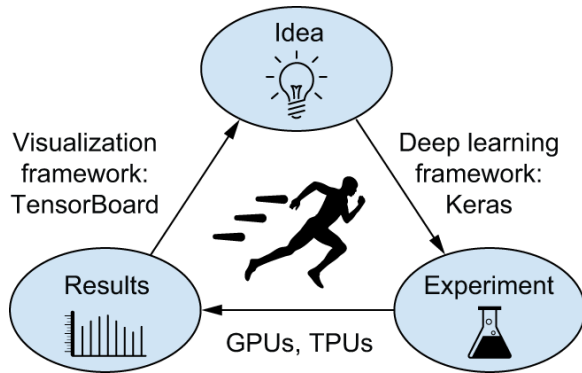
Hình 7.1: Từ Sequential đến Model Subclassing

Spectrum of workflows (Phổ quy trình)

- Keras được thiết kế giống như triết lý của Python: Đa mô hình (multi-paradigm).
- **Người mới (Beginner):** Sử dụng Keras như Scikit-learn (dùng `model.fit()`, `Sequential`).
- **Nhà nghiên cứu (Researcher):** Sử dụng Keras như NumPy/TensorFlow lõi (viết tay mọi thứ từ đầu bằng GradientTape).
- Đặc biệt, các API này **có thể giao tiếp trơn tru với nhau**. Một mô hình `Sequential` có thể được lồng vào một mô hình `Subclassing`.

Vòng lặp Tiến bộ (The Loop of Progress)

- Khoa học là quá trình thử nghiệm lặp đi lặp lại (Iterative Process).
- **Idea:** Khởi tạo ý tưởng.
- **Experiment:** Cài đặt mã nguồn.
- **Evaluate:** Đánh giá kết quả → Rút ra bài học và tạo Idea mới.
- Keras tối ưu hóa trải nghiệm người dùng (UX) để thu ngắn tối đa thời gian lặp trình.



Hình 7.2: Vòng lặp phát triển Mô hình ML

Phương pháp 1: Sequential API

- Cấu trúc dữ liệu dạng **Danh sách (Python List)**.
- Đơn giản nhất, dễ tiếp cận nhất. Các lớp được xếp chồng tuyến tính (lớp sau nối tiếp lớp trước).
- **Hạn chế:** Chỉ áp dụng được cho mô hình 1 đầu vào, 1 đầu ra.
- Không thể áp dụng cho: Mô hình đa đầu vào (Multi-input), đa đầu ra (Multi-output), hay cấu trúc phân nhánh.

Sequential API: Mã nguồn cơ bản

Khởi tạo mô hình Sequential dưới dạng một danh sách (list):

```
1 import keras
2 from keras import layers
3
4 model = keras.Sequential(
5     [
6         layers.Dense(64, activation="relu"),
7         layers.Dense(10, activation="softmax"),
8     ]
9 )
```

Hoặc xây dựng dần dần (incrementally) bằng hàm add():

```
1 model = keras.Sequential()
2 model.add(layers.Dense(64, activation="relu"))
3 model.add(layers.Dense(10, activation="softmax"))
```

Sequential API: Trọng số (Weights) và Shape

Các lớp trong Keras chỉ tạo trọng số (build) khi biết kích thước đầu vào (input shape):

```
1 >>> model.weights # Before build
2 []
3 >>> model.build(input_shape=(None, 3))
4 >>> model.weights # Weights created
5 [<Variable shape=(3, 64) ...>, <Variable shape=(64,) ...>]
```

In bảng tóm tắt cấu trúc:

```
1 >>> model.summary()
2 Model: "sequential_1"
3 ...
```


Sequential API: Khai báo Input Shape trước

Sử dụng đối tượng `Input` để khai báo đầu vào, cho phép mô hình được build tự động ngay từ đầu:

```
1 model = keras.Sequential()
2 model.add(keras.Input(shape=(3,)))
3 model.add(layers.Dense(64, activation="relu"))
4
5 # Model is built automatically, we can call summary now
6 model.summary()
```

Điều này đặc biệt hữu ích khi thiết kế các mạng CNN sâu.

Phương pháp 2: Functional API

- Coi các layer là các **Hàm (Functions)**. Nhận Tensor và trả về Tensor.
- Cung cấp giải pháp linh hoạt hơn nhiều, cho phép xây dựng mạng **Đồ thị đa hướng (Directed Acyclic Graph)**.
- API phổ biến nhất trong thực tế. "Cảm giác như chơi lắp ráp LEGO".
- Cho phép định nghĩa nhiều luồng vào ra, chia nhánh đồ thị.

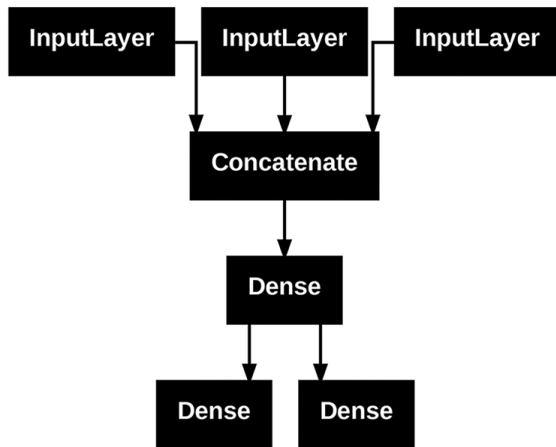
Functional API: Các bước cơ bản

```
1 # 1. Define Input
2 inputs = keras.Input(shape=(3,), name="my_input")
3
4 # 2. Build graph by calling layers like functions
5 features = layers.Dense(64, activation="relu")(inputs)
6
7 # 3. Create outputs
8 outputs = layers.Dense(10, activation="softmax")(features)
9
10 # 4. Initialize Model with inputs and outputs
11 model = keras.Model(inputs=inputs, outputs=outputs, name="
    my_functional_model")
```

Đối tượng `inputs` là một **Symbolic Tensor** (Tensor ký hiệu), chỉ chứa thông tin shape và dtype, không chứa dữ liệu thực.

Functional API: Ví dụ Mạng phân loại vé

- Hệ thống Ticket Classifier.
- **3 Đầu vào (Inputs):**
 - Tiêu đề (title)
 - Nội dung (text_body)
 - Thẻ (tags)
- **2 Đầu ra (Outputs):**
 - Mức độ ưu tiên (priority: 0 đến 1)
 - Phòng ban xử lý (department)



Hình 7.3: Sơ đồ mạng đa luồng

Functional API: Cài đặt hệ thống phân loại vé

```
1 vocabulary_size = 10000
2 num_tags = 100
3 num_departments = 4
4
5 # Inputs
6 title = keras.Input(shape=(vocabulary_size,), name="title")
7 text_body = keras.Input(shape=(vocabulary_size,), name="text_body")
8 tags = keras.Input(shape=(num_tags,), name="tags")
9
10 # Concatenate features
11 features = layers.Concatenate()([title, text_body, tags])
12 features = layers.Dense(64, activation="relu", name="dense_features")(features)
13
14 # Outputs
15 priority = layers.Dense(1, activation="sigmoid", name="priority")(features)
16 department = layers.Dense(num_departments, activation="softmax", name="department")(
    features)
17
18 # Initialize Model
19 model = keras.Model(inputs=[title, text_body, tags], outputs=[priority, department])
```

Functional API: Huấn luyện với Multi-Input/Output

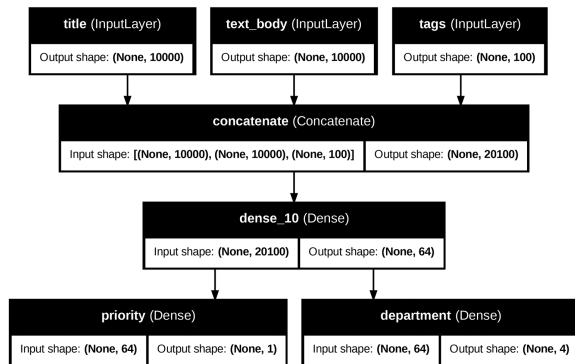
Khi dùng `fit()`, bạn truyền Dữ liệu dạng List hoặc Dictionary:

```
1 model.compile(  
2     optimizer="adam",  
3     loss={  
4         "priority": "mean_squared_error",  
5         "department": "sparse_categorical_crossentropy",  
6     },  
7     metrics={"priority": ["mae"], "department": ["accuracy"]}  
8 )  
9  
10 model.fit(  
11     {"title": title_data, "text_body": text_body_data, "tags": tags_data},  
12     {"priority": priority_data, "department": department_data},  
13     epochs=1  
14 )
```

Sử dụng tên 'name' của Input/Output (Dictionary mode) thường dễ bảo trì hơn so với truyền mảng (List mode).

Functional API: Luồng dữ liệu và Hình dáng (Shapes)

- Có thể vẽ sơ đồ mạng bằng tiện ích `plot_model`.
- Tham số `show_shapes=True` in ra cả kích thước.
- `TensorShape (None, 10000)`: `None` đại diện cho kích thước Lô (Batch size) bất kì.
- Kiểm tra kích thước ở Design Time giúp loại bỏ các lỗi ma trận kích thước lệch pha (Shape Mismatch).

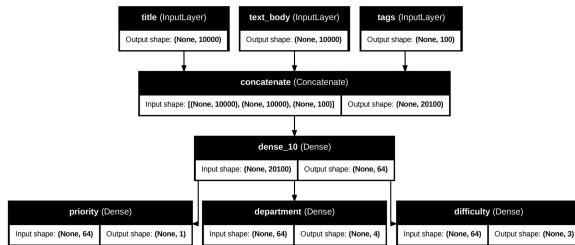


Hình 7.4: Đồ thị truyền tải Shapes

Functional API: Trích xuất Tính năng (Feature Extraction)

- Mô hình chứa các *nút trung gian* (*Intermediate Nodes*). Ta có thể tái sử dụng chúng.
- Trích xuất layer tại mốc số 4 (Dense layer) và bổ sung 1 nhánh difficulty (độ khó).

```
1 features = model.layers[4].output
2 difficulty = layers.Dense(
3     3, activation="softmax", name="difficulty"
4 )(features)
5
6 new_model = keras.Model(
7     inputs=[title, text_body, tags],
8     outputs=[priority, department, difficulty]
9 )
```



Hình 7.5: Mở rộng đồ thị

Phương pháp 3: Model Subclassing

- Phương pháp cấp thấp, linh hoạt nhất.
- Kế thừa trực tiếp Lập trình Hướng đối tượng (OOP) từ lớp `keras.Model`.
- Khai báo các Layer (chứa trọng số) tại hàm tạo `__init__()`.
- Định nghĩa dòng chảy Toán học ngầm (Forward pass) tại hàm `call()`.
- **Ưu điểm:** Cung cấp khả năng kiểm soát tuyệt đối, ví dụ sử dụng vòng lặp `for`, đệ quy bên trong mạng Nơ-ron.

Model Subclassing: Mã nguồn

Viết lại Ticket Classifier dưới dạng Class:

```
1 class CustomerTicketModel(keras.Model):
2     def __init__(self, num_departments):
3         super().__init__() # Always call parent constructor
4         self.concat_layer = layers.Concatenate()
5         self.mixing_layer = layers.Dense(64, activation="relu")
6         self.priority_scorer = layers.Dense(1, activation="sigmoid")
7         self.department_classifier = layers.Dense(num_departments, activation="
softmax")
8
9     def call(self, inputs): # Forward pass logic
10         title = inputs["title"]
11         text_body = inputs["text_body"]
12         tags = inputs["tags"]
13
14         features = self.concat_layer([title, text_body, tags])
15         features = self.mixing_layer(features)
16         priority = self.priority_scorer(features)
17         department = self.department_classifier(features)
18         return priority, department
```

Model Subclassing: Cảnh báo

- **Nhược điểm:** Bạn đang viết "mã byte" dạng hộp đen (Black-box), không còn là đồ thị (Graph) rõ ràng nữa.
- Không thể gọi hàm `summary()` để in ra cấu trúc các nút kết nối.
- Không thể sử dụng `plot_model()` để vẽ ảnh cấu trúc mạng.
- Không thể dùng kĩ thuật **Feature Extraction** do không thể truy xuất `model.layers[i].output`.
- Người lập trình hoàn toàn chịu trách nhiệm cho các lỗi logic bên trong `call()`.

Tùy biến Keras: Khi các hàm có sẵn là không đủ

- API `model.fit()` rất tốt cho hầu hết bài toán thực tế.
- Nhưng với các nhà nghiên cứu, họ thường phát minh ra các hàm Mất mát (Losses) mới, thước đo (Metrics) mới, hoặc những thuật toán Huấn luyện phi tiêu chuẩn (GANs).
- Keras cho phép ghi đè hoàn toàn:
 - Custom Loss
 - Custom Metric
 - Custom Training Step (`train_step()`)
 - Custom Training Loop (Low-level GradientTape)

Tự định nghĩa Hàm Mất mát (Custom Losses)

Đôi khi hàm *Mean Squared Error (MSE)* mặc định là không đủ. Ta có thể viết một hàm (function) đơn giản:

```
1 import tensorflow as tf
2
3 def custom_mean_squared_error(y_true, y_pred):
4     return tf.reduce_mean(tf.square(y_true - y_pred))
5
6 model.compile(optimizer="adam", loss=custom_mean_squared_error)
```

Hoặc kế thừa lớp `keras.losses.Loss`:

```
1 class CustomMSE(keras.losses.Loss):
2     def __init__(self, name="custom_mse", **kwargs):
3         super().__init__(name=name, **kwargs)
4
5     def call(self, y_true, y_pred):
6         return tf.reduce_mean(tf.square(y_true - y_pred))
```

Tự định nghĩa Thước đo (Custom Metrics)

Metrics khác với Loss ở chỗ nó mang **Trạng thái (Stateful)**.

- Loss: Tính cục bộ trên mỗi Batch (Lô).
- Metric: Phải cộng dồn trên nhiều Batch và tính trung bình trong suốt 1 Epoch.
- Kế thừa lớp `keras.metrics.Metric` yêu cầu 3 hàm:
 - 1 `update_state()`: Cộng dồn kết quả đếm cho mỗi Batch.
 - 2 `result()`: Tính toán và trả về kết quả hiển thị cho người dùng.
 - 3 `reset_state()`: Xóa sạch bộ đếm về 0 khi bắt đầu một Epoch mới.

Custom Metrics: Cài đặt thực tế (Root Mean Squared Error)

```
1 class RootMeanSquaredError(keras.metrics.Metric):
2     def __init__(self, name="rmse", **kwargs):
3         super().__init__(name=name, **kwargs)
4         # Declare state variables
5         self.mse_sum = self.add_weight(name="mse_sum", initializer="zeros")
6         self.total_samples = self.add_weight(name="total_samples", initializer="
zeros")
7
8     def update_state(self, y_true, y_pred, sample_weight=None):
9         mse = tf.reduce_mean(tf.square(y_true - y_pred))
10        self.mse_sum.assign_add(mse)
11        self.total_samples.assign_add(1.0)
12
13    def result(self):
14        # Return sqrt of accumulated mean MSE
15        return tf.sqrt(self.mse_sum / self.total_samples)
16
17    def reset_state(self):
18        self.mse_sum.assign(0.)
19        self.total_samples.assign(0.)
```

Viết lại Vòng lặp Huấn luyện với GradientTape

Thay vì gọi `model.fit()`, ta có thể sử dụng `tf.GradientTape` để tự điều khiển phép tính **Đạo hàm riêng (Backpropagation)**:

$$\Delta W = -\eta \cdot \frac{\partial L}{\partial W} \quad (1)$$

Quy trình 4 bước của 1 Batch (Lô dữ liệu):

- ➊ Mở không gian bộ nhớ (Tape), truyền dữ liệu qua Mạng.
- ➋ Tính toán Loss L .
- ➌ Dùng Tape truy xuất Gradient $\frac{\partial L}{\partial W}$ của các Trọng số.
- ➍ Gửi Gradient cho Trình tối ưu hóa (Optimizer) để cập nhật Trọng số W .

Custom Training Loop: Mã nguồn GradientTape

Đây là kiến thức quan trọng nhất dành cho một Chuyên gia Học sâu.

```
1 optimizer = keras.optimizers.Adam(learning_rate=1e-3)
2 loss_fn = keras.losses.SparseCategoricalCrossentropy()
3
4 for epoch in range(3): # Epoch loop
5     for step, (x_batch, y_batch) in enumerate(train_dataset): # Batch loop
6         # 1. Open Tape
7         with tf.GradientTape() as tape:
8             # 2. Feed-forward and calculate loss
9             logits = model(x_batch, training=True)
10            loss_value = loss_fn(y_batch, logits)
11
12            # 3. Compute Gradients
13            gradients = tape.gradient(loss_value, model.trainable_weights)
14
15            # 4. Apply Gradients (Update weights)
16            optimizer.apply(gradients, model.trainable_weights)
```

Ghi đè hàm train_step()

Để kết hợp vòng lặp thủ công với tính năng của fit() (hiển thị thanh tiến trình), bạn có thể ghi đè train_step():

```
1 class CustomModel(keras.Model):
2     def train_step(self, data):
3         inputs, targets = data
4         with tf.GradientTape() as tape:
5             predictions = self(inputs, training=True)
6             loss = self.compute_loss(y=targets, y_pred=predictions)
7
8         gradients = tape.gradient(loss, self.trainable_weights)
9         self.optimizer.apply_gradients(gradients, self.trainable_weights)
10
11     # Update Metrics
12     for metric in self.metrics:
13         if metric.name == "loss":
14             metric.update_state(loss)
15         else:
16             metric.update_state(targets, predictions)
17     return {m.name: m.result() for m in self.metrics}
```

Bạn vẫn có thể dùng 'model.fit()' và tận hưởng mọi Callback.

Sức mạnh của Callbacks

- Huấn luyện mô hình Deep Learning có thể tốn hàng giờ hoặc nhiều tuần.
- Bạn không thể cứ ngồi nhìn màn hình suốt quá trình đó để can thiệp kịp thời.
- **Callbacks** là các "hook" (chốt chặn) được kích hoạt tự động vào các thời điểm cụ thể (Đầu/cuối Epoch, Đầu/cuối Batch).
- **Ứng dụng:** Tự dừng sớm, Lưu trọng số, Thay đổi tốc độ học...

Các Callbacks tích hợp: EarlyStopping & ModelCheckpoint

EarlyStopping: Tự động dừng huấn luyện khi val_loss không còn giảm (Ngăn Overfitting).

ModelCheckpoint: Tự động sao lưu Trọng số Tốt nhất xuống ổ cứng.

```
1 callbacks_list = [  
2     keras.callbacks.EarlyStopping(  
3         monitor="val_accuracy",  
4         patience=2, # Stop if no improvement after 2 epochs  
5     ),  
6     keras.callbacks.ModelCheckpoint(  
7         filepath="my_checkpoint.keras",  
8         monitor="val_loss",  
9         save_best_only=True, # Only save model with lowest val_loss  
10    )  
11 ]  
12 model.fit(..., callbacks=callbacks_list, ...)
```

Tùy biến Callback (Custom Callback)

Bạn có thể tự tạo Callback bằng cách kế thừa `keras.callbacks.Callback`. Các hàm sự kiện bạn có thể ghi đè:

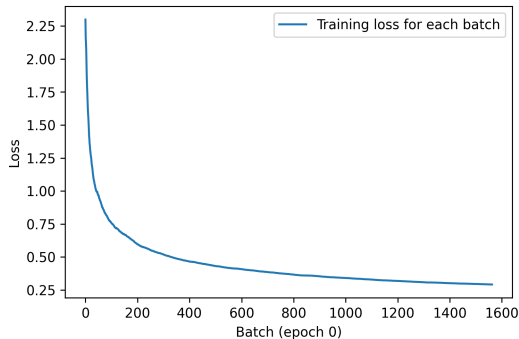
```
1 class MyCustomCallback(keras.callbacks.Callback):
2     def on_epoch_begin(self, epoch, logs=None):
3         print(f"Start epoch {epoch}")
4
5     def on_epoch_end(self, epoch, logs=None):
6         print(f"End epoch {epoch}")
7
8     def on_batch_begin(self, batch, logs=None): ...
9     def on_batch_end(self, batch, logs=None): ...
10    def on_train_begin(self, logs=None): ...
11    def on_train_end(self, logs=None): ...
```

Custom Callback: Ví dụ ghi lại Loss theo Batch

```
1 import matplotlib.pyplot as plt
2
3 class LossHistory(keras.callbacks.Callback):
4     def on_train_begin(self, logs=None):
5         self.per_batch_losses = [] # Store loss for every batch
6
7     def on_batch_end(self, batch, logs=None):
8         self.per_batch_losses.append(logs.get("loss"))
9
10    def on_train_end(self, logs=None):
11        plt.clf()
12        plt.plot(range(len(self.per_batch_losses)), self.per_batch_losses, label="
Training loss")
13        plt.xlabel(f"Batches (Total: {len(self.per_batch_losses)})")
14        plt.ylabel("Loss")
15        plt.legend()
16        plt.savefig("loss_history_callback_example.png")
```

Kết quả sinh ra bởi Custom Callback

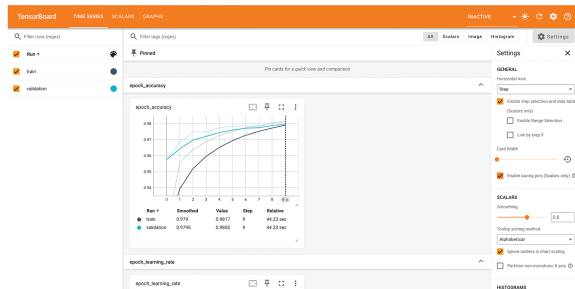
- Mật độ dày đặc của biểu đồ cho thấy sự thay đổi Loss ở cấp độ Lô (Batch-level) (Thay vì mức Epoch).
- Đây là một phương pháp kiểm tra (sanity-check) xuất sắc để xem dữ liệu có thực sự "nhảy" với bước học của mạng Nơ-ron hay không.



Hình 7.6: Đồ thị Loss sinh ra cuối quá trình

Bảng điều khiển TensorBoard

- Mắt thường không thể đọc hàng triệu con số in trên Terminal.
- **TensorBoard** chạy một Web Server nội bộ (chạy trên localhost:6006), theo dõi và trực quan hóa thời gian thực (Real-time).
- Là công cụ chuẩn mực của giới chuyên gia để phân tích sâu hoạt động mạng.



Hình 7.7: Giao diện Giám sát TensorBoard

Kích hoạt TensorBoard trong Keras

Rất đơn giản, bạn chỉ cần dùng Callback TensorBoard có sẵn:

```
1 callbacks = [  
2     keras.callbacks.TensorBoard(  
3         log_dir="/full_path_to_your_logs",  
4     )  
5 ]  
6 model.fit(..., callbacks=callbacks)
```

Sau khi model chạy, mở Terminal và gọi:

```
1 $ tensorboard --logdir /full_path_to_your_logs
```

Sau đó truy cập <http://localhost:6006> trên trình duyệt.

Các tính năng cốt lõi của TensorBoard

- **Metrics Viewer:** So sánh Metrics (Loss, Accuracy) của hàng chục thí nghiệm khác nhau cùng lúc (để Tune Hyperparameters).
- **Histograms:** Vẽ biểu đồ phân phối trọng số/kích hoạt để kiểm tra Gradient có bị Triệt tiêu (Vanishing) hay Bùng nổ (Exploding) hay không.
- **Graphs:** Vẽ lại Đồ thị Kiến trúc mô hình đa luồng.
- **Embeddings:** Chiếu (Project) không gian vector hàng ngàn chiều xuống 2D/3D (VD: PCA, t-SNE) để xem sự phân cụm của dữ liệu.

Keras không chỉ là thư viện dành cho người mới, nó là một Vũ khí Chuyên sâu:

- Chọn đúng công cụ:
 - **Sequential**: Đơn giản (Mức 1).
 - **Functional API**: Đa luồng đồ thị - Khuyến dùng hầu hết thời gian (Mức 2).
 - **Model Subclassing**: Tự điều khiển luồng OOP (Mức 3).
- Khả năng tùy biến vô hạn: Có thể tự viết lại **Custom Loss**, **Custom Metric**, và sử dụng **GradientTape** thay vì `fit()`.

Tổng kết Chương 7 (tiếp)

- Giao thoa Workflow: Kết hợp Vòng lặp Tape thủ công bên trong hàm `train_step()` để có ưu điểm của cả hai thế giới.
- Tự động hóa quá trình chạy thực nghiệm (Experiment) với hệ thống **Callbacks** (EarlyStopping, Checkpoint).
- Giám sát học thuật chuyên nghiệp: Trực quan hóa mọi thứ trực tuyến với Web Server **TensorBoard**.

Bây giờ bạn đã đủ kiến thức để triển khai bất kỳ kỹ thuật học sâu phức tạp nào trong các chương tiếp theo!